

Are Small LLMs Ready for Coding Agents?

Spoiler: yes — if the harness does its share of the work

Denis Akhyyarov · Agentic AI Summit, UC Berkeley · Aug 1, 2026

One agent. One file. One laptop. ~7 tokens/sec. Let's see how far that gets us.

The harness is half the model

NanAgent: single Python file, no frameworks. Preflight → Plan → Execute → Replan.

What it took to make a 4B-active model finish real coding tasks:

Harness trick	Why the small model needs it
<code>edit</code> action (find/replace)	Full-file <code>write</code> blows the 256-token budget
Mechanical JSON repair	~60% of first edit attempts emit broken JSON
Error- class retry policy	"File not found" needs a read, not more thinking
Task-local replan	Fix one failed task in ~5s instead of a ~110s full replan
Slim executor state (~200 tok)	16K context, 7 tok/s — every token is wall-clock

Each of these was worth more than a model-size upgrade on our tasks.

The contestants: three sizes of "small"

Model	Shape	Runs on	Vibe
Gemma 4 E4B	MoE 12B / 4B active	16GB M1 laptop, ~7 tok/s	The scrappy one
Gemma 4 26B-A4B	MoE 26B / 4B active	Hosted, ~1–2s per step	The sweet spot
Qwen3.6-27B / 35B-A3B	Dense 27B / MoE 3B active	Single 24GB GPU	The new open coder

- Same harness, same tests, three size classes — size is the only variable
- Qwen3.6-27B: arguably the best open coder on a single consumer GPU right now
- (RIP Qwen 3.5 9B — always-on `<think>` blocks fought the harness and lost)

Full app generation mode: the receipts

Agent harness suite, easy → hard (build/fix/replan tasks), 3 trials each:

Suite	E4B local (median)	26B hosted (median)	Pass
Easy	20–119s	2.5–5s	Both 9/9
Medium	29–609s	2–39s	Both 9/9
Hard	80–904s	9.5–43s	Both 9/9

- **Same completion rate.** The small model gets there — it just takes the scenic route (4–24× slower)
- Where the time goes: failed edits → thinking retries → re-reads. **~60% was harness-fixable**
- Full app generation ("build me a working X, compiled and running") is the same loop, more tasks — final validation catches "all tasks passed but the app doesn't run"

Benchmarks that actually smell like app coding

(SWE-bench is great — it's also patch-fixing in mature repos, not *building apps*)

- **WebDev Arena** (LMArena) — humans blind-vote on generated web apps. Elo, not unit tests. Frontier models lead; open ~30B models are climbing into the same bracket
- **WebCoderBench** — 1,572 *real user requests* for web apps, interpretable metrics. Messy prompts included, which is exactly how users talk
- **LiveCodeBench** — continuously refreshed problems = contamination-free. The honest "can it code at all" floor

Pattern across all three: small open models trail frontier by one league, not one era.

Benchmarks lie a little. Run the app.

Our spot-check ritual for generated apps — 10 minutes, no leaderboard needed:

1. **Does it run?** Not "does it compile" — `npm start` / `python app.py` and click around
2. **The second feature.** First feature is always fine; feature #2 is where small models quietly drop state, break routes, forget the DB schema
3. **Error paths.** Empty input, missing file, wrong port — small models write the happy path and *only* the happy path
4. **Read the diff like a reviewer**, not a fan

Small-model failures are **boring and mechanical** (truncation, path typos, lost context) → exactly what harnesses catch. Frontier failures are subtle design mistakes → much harder to guard.

So... are they ready?

Ready today — bounded coding-agent work: build-fix-test loops, scaffolding, file surgery, CI bots, private/offline/on-device agents. *If* your harness has typed errors, cheap recovery, and real validation.

Not yet — one-shot full apps at frontier quality, long-horizon refactors, >16K-context reasoning. Frontier models still win the *leap*; small models win the *loop*.

The bet: capability that needed a frontier model 18 months ago runs on a laptop today. Harness work is the *durable* asset — every trick transfers upward, and models only get smaller-better.

Code, benchmarks & the full pain journal: github.com/den-run-ai/askme — thanks!